AISSMS
INSTITUTE OF INFORMATION TECHNOLOGY
ADDING VALUE TO ENGINEERING

Department of Artificial Intelligence & Data Science

Lab Manual

DataBase Management Systems Laboratory

(ADPCC403)

Prepared by

Dr.S.S.Sheikh

Mrs. Kalpana Sonwal

SY BTech AI & DS

Semester II

Academic Year 2025-26

Vision

Mission

Program Education Objectives(PEOs)

To be amongst the top artificial intelligent & data scienc
catering to the changing needs of the industry and

To foster an environment to provide intelligent s
applicable for multidisciplinary needs of industry &

To promote career development with ethical resp

Program Specific Outcomes (PSOs)

PEO1: Graduates will be able to analyse, formulate and fun
in a multi-disciplinary context to address industrial proble

PEO2: Graduates will be able to work collaboratively with
and ethical responsibilities to provide innovative needs of

PEO3: Graduates will excel in their careers by a
technologies.

PSO1 Problem Solving and Programming Skills: Graduat
apply programming skill to identify, modify and test algo

Program Outcomes (POs)

Graduates will be able to

1. Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems. **[Engineering knowledge]**
2. Identify, formulate, research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences. **[Problem analysis]**
3. Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations. **[Design/development of solutions]**
4. Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions. **[Conduct investigations of complex problems]**
5. Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations. **[Modern tool usage]**
6. Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice. **[The engineer and society]**
7. Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development. **[Environment and sustainability]**
8. Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice. **[Ethics]**
9. Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings. **[Individual and team work]**
10. Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions. **[Communication]**
11. Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments. **[Project management and finance]**
12. Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change. **[Life-long learning]**

List of Experiments

	Lab No. Name of Subject : DBMS Lab Class: SY Btech.AI&DS				
Sr. No.	**Experiment Name**	**Equipment/ Software required**	**CO**	**PO**	**PSO**
	Group A: Database Programming SQL,PL/SQL				
1	**SQL Queries:** • Design and Develop SQL Data definition languages (DDL) commands of base tables and views. • DDL commands: CREATE , ALTER ,DROP , RENAME , TRUNCATE • Write at least10 SQL queries for suitable database application using SQL DDL statements.		403.1	1,2,11	
2	**SQL Queries:** • Data manipulation language (DML) of base tables and views. • DML commands: INSERT , UPDATE , DELETE , SELECT Write at least10 SQL queries for suitable database application using SQL DML statements	Open source operating system- Fedora 21. MySQL	403.1	1,2,11	1, 2
3	**Perform the following:** a. Viewing all databases, Creating a Database, Viewing all Tables in a Database, Creating Tables (With and Without Constraints), Inserting/Updating/Deleting Records in a Table, Saving (Commit) and Undoing (rollback)	Open source operating system- Fedora 21. MySQL	403.1	1,2,11	1, 2
4	For a given set of relation schemes, create tables and perform the following a. Simple Queries, Simple Queries with Aggregate functions, Queries with Aggregate b. functions (group by and having clause), Queries involving- Date Functions, String	Oracle 11g	403.1	1,2,11	1, 2

	c. Functions , Math Functions d. Join Queries- Inner Join, Outer Join e. Sub queries- With IN clause, With EXISTS clause				
5	**PL/SQL Programming** i. Creating stored procedures, functions and packages ii. Error handling and Exception iii. Triggers and auditing triggers	Oracle 11g	403.1	1,2,11	1,2
6	**Cursors(All types)** Write a PL/SQL block that will display the name, dept no, salary of fist highest paid employees. OR Write a PL/SQL block that will display the employee details along with salary using cursors.	Oracle 11g	403.1	1,2,11	1,2
7	Write a Mongo DB query to display the fields restaurant_id, name, borough and cuisine for all the documents in the collection restaurant.	Oracle 11g	403.1	1,2,11	1,2
8	Design and Develop MongoDB queries using aggregation and indexing with suitable example.	Oracle 11g	403.1	1,2,11	1,2
Group C :- Mini Project : Database Project					
1	1.Develop an application with following details: • Front End: Python/Java/PHP/Perl/Ruby/.NET/ or any other language • Backend : MongoDB/ MySQL/ Oracle / or any standard SQL / NoSQL database 2. Test and validate application using Manual/Automation testing. 3.Student should develop application in group of 2-3 students and submit the Project Report which will consist of documentation related to different phases of Software Development Life Cycle: • Title of the Project, Abstract, Introduction	Front End: Python/Java/PHP/Perl/ Ruby/.NET/ or any other language Backend : MongoDB/ MySQL/ Oracle / or any standard SQL / NoSQL database	403.1	1,2,3,4,5,9,10,11	1,2,3

	• Software Requirement Specification (SRS) • Conceptual Design using ER features, Relational Model in appropriate Normalize form • Graphical User Interface, Source Code • Testing document • Conclusion.				

Assignment No. 1

TITLE :

SQL Queries: Design and Develop SQL Data definition languages (DDL) commands of base tables and views.

- DDL commands: CREATE , ALTER ,DROP , RENAME , TRUNCATE
- Write at least10 SQL queries for suitable database application using SQL DDL statements.

OBJECTIVE :

To design and develop SQL Data Definition Language (DDL) commands for creating, modifying, and deleting base tables and views in a relational database management system (RDBMS). To learn to utilize various DDL commands such as CREATE, ALTER, DROP, RENAME, and TRUNCATE to define the structure of base tables and views effectively.

PROBLEM STATEMENT:

(Exercise on updating records in table)

Create Client_master with the following fields (ClientNO, Name, Address, City, State, bal_due)

(a) Create table Insert ten records

(b) Find the names of clients whose bal_due> 5000 .

(c) Change the bal_due of ClientNO " C123" to Rs. 5100

(d) Change the name of Client_master to Client12 .

(e) Display the bal_due heading as "BALANCE" Client master table.

(f) Change the name of the relation to Cutomer_info

(g) Change the data type of the bal_due to Float and display with the name "Balance"

(h) Delete all the records but keep the table structure as it is

(i) Add the column Pin code in the table

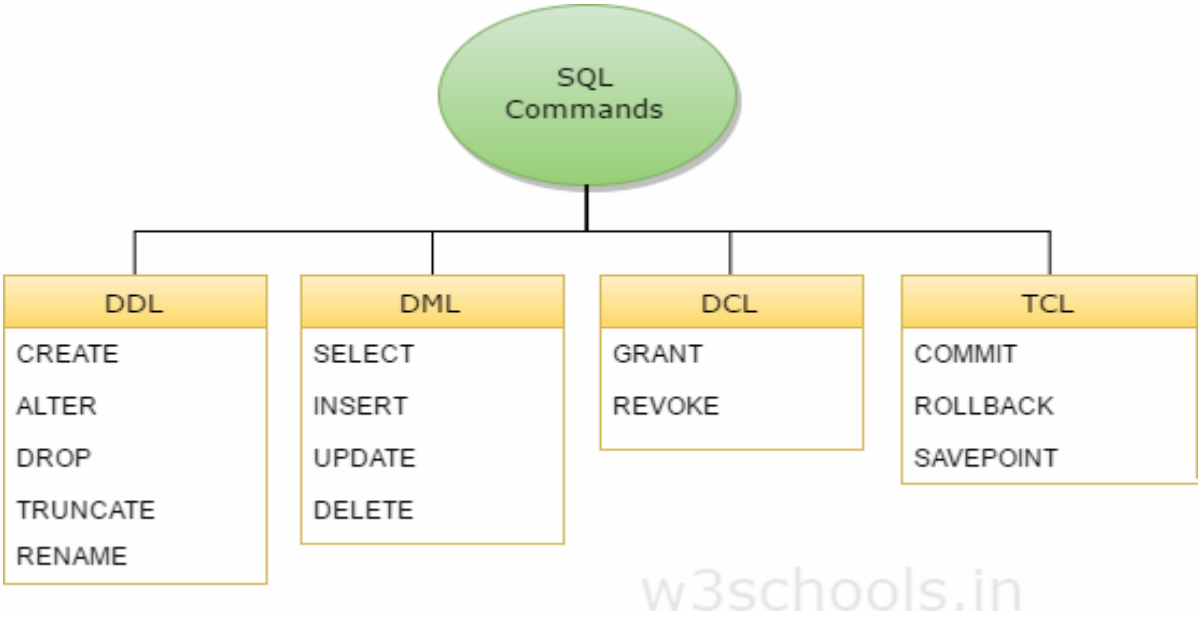
(j) Display the list of clients whose city is Pune

THEORY:

Data Definition Language

The data definition language is used to create an object, alter the structure of an object and also drop already created object. The Data Definition Languages used for table definition can be classified into following:

- Create table command
- Alter table command
- Truncate table command



<u>CREATION OF TABLES:</u>

CREATE TABLE:

Table is a primary object of database, used to store data in form of rows and columns. It is created using following command:

Syntax: CREATE TABLE tablename (column_name data_ type constraints, …)

CREATING TABLES:

Example:

mysql> create table Bus (Bus_No varchar(5), source varchar(20), destination varchar(20), daysperweek int); Table Created.

Above definition will create simple table. Still there are more additional option related with create table for the object-relation feature we will discuss it afterwards.

Desc command:

Describe command is external command of Oracle. The describe command is used to view thestructure of table as follows.

Desc <table name>

mysql>

desc Bus**;**

Field	Type	Null	Key	Default	Extra
Bus_No	varchar(5)	YES		NULL	
source	varchar(20)	YES		NULL	
destination	varchar(20)	YES		NULL	

Creating Passenger table:

Mysql>create table passenger(pnrno integer,ticketno integer,name varchar(20),age int,sex char,ppno

integer); Mysql>desc passenger;

Field	Type	Null	Key	Default	Extra
pnrno	int	YES	MUL	NULL	
ticketno	int	YES		NULL	
name	varchar(20)	YES		NULL	
age	int	YES		NULL	
sex	char(1)	YES		NULL	
ppno	int	YES		NULL	

<u>Reservation Table:</u>

mysql > create table Reservation (PNR_NO integer(9), No_of_seats integer(8), Address varchar(50), Contact_No Bigint(12), Status varchar(10));

desc Reservation;

Field	Type	Null	Key	Default	Extra
PNR_NO	int	YES		NULL	
No_of_seats	int	YES		NULL	
Address	varchar(50)	YES		NULL	
Contact_No	bigint	YES		NULL	
status	varchar(10)	YES		NULL	

<u>Cancellation Table:</u>

mysql > create table Cancellation (PNR_NO integer (9), No_of_seats integer (8), Address varchar (50), Contact_No integer (12), Status char (3));

Table created.

SQL> desc Cancellation;

```
Field    Type     Null     Key      Default Extra
PNR_NO   int      YES              NULL
No_of_seats       int      YES              NULL
Address varchar(50)        YES              NULL
Contact_No        int      YES              NULL
Status   char(3) YES               NULL
```

Ticket Table:

mysql >create table Ticket(Ticket_No integer(9) primary key, age int, sex char(4) Not null, source varchar(2), destination varchar(20), dep_time varchar(4));
Table

created;

mysql > desc

Ticket;

```
Field    Type     Null     Key      Default Extra
Ticket_No         int      NO       PRI     NULL
age      int      YES              NULL
sex      char(4) NO                NULL
source   varchar(2)        YES              NULL
destination       varchar(20)      YES              NULL
dep_time          varchar(4)       YES              NULL
```

ALTER TABLE :

To ADD a column:

SYNTAX: ALTER TABLE <TABLE NAME>ADD (<NEW COLUMN NAME><DATA TYPE>(<SIZE>), <NEW COLUMNNAME><DATA TYPE>(<SIZE>));

Example:

mysql > alter table Reservation add column fare integer; mysql > desc Reservation;

Field	Type	Null	Key	Default	Extra
PNR_NO	int	YES		NULL	
No_of_seats	int	YES			NULL
Address	varchar(50)	YES			NULL
Contact_No	int	YES			NULL
Status	char(3)	YES		NULL	
fare	int	YES		NULL	

To DROP a column:

SYNTAX: ALTER TABLE <TABLE NAME>DROP COLUMN <COLUMN NAME>;

Example:

mysql >alter table Reservation drop column fare ; mysql > desc Reservation;

To MODIFY a column:

SYNTAX: ALTER TABLE <TABLE NAME>MODIFY COLUMN <COLUMN NAME>

<NEW DATATYPE>(<NEW SIZE>);

Example:

mysql >alter table Reservation modify column status varchar(10);

mysql >desc Reservation;

TO ADD FOREIGN KEY TO THE EXISTING TABLE

mysql > ALTER TABLE passenger ADD FOREIGN KEY (pnrno) REFERENCES Reservation (PNR_NO); Table altered.

desc passenger

mysql > ALTER TABLE Cancellation ADD FOREIGN KEY (PNR_NO) REFERENCES **Reservation (PNR_NO);**

TRUNCATE TABLE:

If there is no further use of records stored in a table and the structure is required then only data can be

deleted using truncate command. Truncate command will delete all the records permanently of specified table as follows.

Truncate table <table
name> EXAMPLE: try
your own Query

RENAME A TABLE

Rename command is used to give new names for existing tables.
RENAME table old tablename TO new tablename;

Example:
MYSQL>RENAME table passenger TO Passenger;

View:

SQL allows a ―virtual relation‖ to be defined by a query, and the relation conceptually contains the result of the query. The virtual relation is not precomputed and stored, but instead is computed by executing the query whenever the virtual relation is used. Any such relation that is not part of the logical model, but is made visible to a user as a virtual relation, is called a view.

Create View: We define a view in SQL by using the create view command.

create view v as <query expression>;

where <query expression> is any legal query expression. The view name is represented by v.

Alter View: The CREATE VIEW statement creates a new view, or replaces an existing view if the OR REPLACE clause is given. If the view does not exist, CREATE OR REPLACE VIEW is the same as CREATE VIEW.

Drop view: Use the DROP VIEW statement to remove a view or an object view from the database. You can change the definition of a view by dropping and re-creating it.

Syntax:

Drop view viewname;

Conclusion:

Assignment No. 2

TITLE:

SQL Queries:

- Data manipulation language (DML) of base tables and views.
- DML commands: INSERT , UPDATE , DELETE , SELECT

Write at least10 SQL queries for suitable database application using SQL DML statements

Objectives: To understand and effectively utilize the Data Manipulation Language (DML) commands for performing operations on base tables and views in a relational database management system (RDBMS). Learn to apply DML commands such as INSERT, UPDATE, DELETE, and SELECT to manipulate data within the database.

Problem Statement:

EMPLOYEES (Employee_Id, First_Name, Last_Name, Email, Phone_Number, Hire_Date,

Job_Id, Salary, Commission_Pct, Manager_Id, Department_Id)

(a) Find out the employee id, names, salaries of all the employees

(b) List out the employees who works under manager 100

(c) Find the names of the employees who have a salary greater than or equal to 4800

(d) List out the employees whose last name is 'AUSTIN'

(e) Find the names of the employees who works in departments 60,70 and 80

(f) Display the unique Manager_Id.

(g) Delete the record of the Employee id 102

(h)change the salary of the employee "John" to 50000

(i) List the employees who has joined after 1st Jan 2010

(j) Display the names of the employees whose first name starts with "A"

Data Manipulation Language (DML) Statements:

Data manipulation language (DML) statements access and manipulate data in existing schema objects. These statements do not implicitly commit the current transaction. The SELECT statement is a limited form of DML statement in that it can only access data in the database. It cannot manipulate data in the database, although it can operate on the accessed data before returning the results of the query.

The data manipulation language statements are: INSERT, UPDATE, DELETE, SELECT

DML COMMANDS

1. INSERTING DATA IN TO TABLE

Insert command is used to insert
rows into the table. SYNTAX:
INSERT INTO table name values (columnname1, column name2,…., column name n)

INSERTION of Data can also be done by the
following Syntax: SYNTAX
INSERT IN TO tablename (columnname1, columnname2,….columnname n) VALUES(Value1,Value2,..Value n);

Inserting values into "Bus" table:

mysql > insert into Bus values('w1234','hyderabad', 'tirupathi',4);

mysql >insert into Bus values ('p2345','hyderabad', 'Banglore',3);

mysql >insert into Bus values ('9w01','hyderabad','Kolkata',4);

Inserting values into " RESERVATION" table:

mysql >insert into Reservation values(1,2,'masabtank',9009897812,'confirm');
mysql>insert into Reservation values(1,2,'masabtank',9009897812,'confirm');

Inserting values into "PASSENGER" table:

mysql >insert into Passenger values (1, 1,'SACHIN', 12,'m', 1234);
mysql >insert into Passenger values (2, 2,'rahul', 34,'m', 3456);
mysql >insert into Passenger values(3,3,'swetha',24,'f',8734);
mysql >insert into Passenger values(5,5,'Arun',24,'m',7387);
mysql >insert into Passenger values(6,6,'Aruna',25,'f',7389);
mysql >insert into Passenger values(4,3,'rohith',24,'m',734);

UPDATE

This SQL command is used to modify the values in an existing table.

mysql >**UPDATE** table name

SET column1= expression1, column2= expression 2,...

WHERE some column=some value;

An expression consists of either a constant (new value), an arithmetic or string operation or an SQL query. Note that the new value to assign to <column> must matching data type.

An update statement used without a where clause results in changing respective attributes of all tuples in the specified table.

EXAMPLE:

mysql >update Passenger set age='43' where pnrno='2'; TEST OUTPUT:

DELETE

In order to delete rows from a table we use this command

mysql >**DELETE** FROM tablename WHERE condition;

EX: delete from Passenger
where pnrno='3'; 1 row deleted.

SQL> select * from Passenger;

TO RETRIEVE / DISPLAY DATA FROM TABLES:

a. Select command is used to select values or data from table SYNTAX

SELECT * FROM TABLENAME;

Example:

SQL> select * from Passenger;

b. Elimination of duplicates from the select statement

SQL> **SELECT DISTINCT** columnname 1, columnname 2,…. columnname n FROM tablename;

Conclusion

Assignment No. 3

TITLE:

Perform the following:
a. Viewing all databases, Creating a Database, Viewing all Tables in a Database, Creating Tables (With and Without Constraints), Inserting/Updating/Deleting Records in a Table, Saving (Commit) and Undoing (rollback)

OBJECTIVE: to understand and perform essential database management operations using SQL commands.Learn to interact with databases, create and manage databases and tables, manipulate data within tables, and manage transactions effectively.

Problem Statement:

Create Teacher table with the following fields(Name, DeptNo, Date of joining, DeptName, Location, Salary)

(a) Insert ten records

(b) Give Increment of 25% salary for Mathematics Department .

(c) Perform Rollback command

(d) Give Increment of 15% salary for Commerce Department

(e) Perform commit command

(f) Add the constraint not null on the Date of Joining

(g) Change the datatype of JOB_ID from char to varchar2

(h) Add a new column; HIREDATE to the existing relation and add the constraint not null on it

(i) Modify the column width of the job field of Teacher table

(j) Delete the record of the commerce department

Theory:

ADDING/REMOVING CONSTRAINTS TO A TABLE

ALTER TABLE statement can be used to add constraints to your existing table by using it in following manner:

TO ADD PRIMARY KEY CONSTRAINT

ALTER TABLE <table name>

ADD PRIMARY KEY (Column name);

e.g. to add PRIMARY KEY constraint on column ECODE of table EMPLOYEE , the command is : ALTER TABLE EMPLOYEE ADD PRIMARY KEY (ECODE)

TO ADD FOREIGN KEY CONSTRAINT ALTER TABLE

ADD FOREIGN KEY (Column name) REFERENCES Parent Table (Primary key of Parent Table);

REMOVING CONSTRAINTS –

To remove primary key constraint from a table, we use ALTER TABLE command as :
ALTER TABLE <Table name>

DROP PRIMARY KEY ; - To remove foreign key constraint from a table, we use ALTER TABLE command as : ALTER TABLE <Table name>
DROP FOREIGN KEY

ENABLING/DISABLING CONSTRAINTS Only foreign key can be disabled/enabled in MySQL. To disable foreign keys : SET FOREIGN_KEY_CHECKS = 0 ; To enable foreign keys : SET FOREIGN_KEY_CHECKS = 1 ;

INTEGRITY CONSTRAINTS/CONSTRAINTS - A constraint is a condition or check applicable on a field(column) or set of fields(columns). - Common types of constraints include

S.No.	Constraints	Description
1	NOT NULL	Ensures that a column cannot have NULL value
2	DEFAULT	Provides a default value for a column when none is specified
3	UNIQUE	Ensures that all values in a column are different
4	CHECK	Makes sure that all values in a column satisfy certain criteria
5	PRIMARY KEY	Used to uniquely identify a row in the table
6	FOREIGN KEY	Used to ensure referential integrity of the data

NOT NULL CONSTRAINT

By default, a column can hold NULL. It you not want to allow NULL value in a column, then NOT NULL constraint must be applied on that column.

E.g.

CREATE TABLE Customer

(SID integer NOT NULL ,

Last_Name varchar(30)

First_Name varchar(30)

NOT NULL

) ;

,

Columns SID and Last_Name cannot include NULL, while First_Name can include NULL.

An attempt to execute the following SQL statement,

INSERT INTO Customer

VALUES (NULL , ‘Kumar’ , ‘Ajay’);

will result in an error because this will lead to column SID being NULL, which violates the NOT NULL constraint on that column

DEFAULT CONSTARINT

The DEFAULT constraint provides a default value to a column when the INSERT INTO statement does not provide a specific value. E.g.

CREATE TABLE Student

(Student_ID integer ,

Name varchar(30) ,

Score integer DEFAULT 80);

When following SQL statement is executed on table created above:

INSERT INTO Student no value has been provided for score field.

VALUES (10 , ‘Ravi’);

Then table Student looks like the following:

Student_ID	Name	Score
10	Ravi	**80**

score field has got the default value

UNIQUE CONSTRAINT

- The UNIQUE constraint ensures that all values in a column are distinct. In other words, no two rows can hold the same value for a column with UNIQUE constraint.

e.g.

CREATE TABLE Customer

(SID integer Unique ,

Last_Name varchar(30) ,

First_Name varchar(30)) ;

Column SID has a unique constraint, and hence cannot include duplicate values. So, if the table already contains the following rows :

SID	Last_Name	First_Name
1	Kumar	Ravi
2	Sharma	Ajay
3	Devi	Raj

The executing the following SQL statement,

INSERT INTO Customer

VALUES (‘3’ , ‘Cyrus‘ , ‘Grace’) ;

will result in an error because the value 3 already exist in the SID column, thus trying to insert another row with that value violates the UNIQUE constraint.

CHECK CONSTRAINT

- The CHECK constraint ensures that all values in a column satisfy certain conditions. Once defined, the table will only insert a new row or update an existing row if the new value satisfies the CHECK constraint.

e.g.

CREATE TABLE Customer

(SID integer CHECK (SID > 0),

Last_Name varchar(30) ,

First_Name varchar(30)) ;

So, attempting to execute the following statement :

INSERT INTO Customer

VALUES (-2 , ‘Kapoor’ , ‘Raj’);

will result in an error because the values for SID must be greater than 0

PRIMARY KEY CONSTRAINT

- A primary key is used to identify each row in a table. A primary key can consist of one or more fields(column) on a table. When multiple fields are used as a primary key, they are called a composite key.

- You can define a primary key in CREATE TABLE command through keywords PRIMARY KEY.

e.g.

CREATE TABLE Customer

(SID integer NOT NULL PRIMARY KEY,

Last_Name varchar(30) ,

First_Name varchar(30)) ;

Or

CREATE TABLE Customer

(SID integer,

Last_Name varchar(30) ,

First_Name varchar(30),

PRIMARY KEY (SID)) ;

- The latter way is useful if you want to specify a composite primary key, e.g.

CREATE TABLE Customer

(Branch integer NOT NULL,

SID integer NOT NULL ,

Last_Name varchar(30) ,

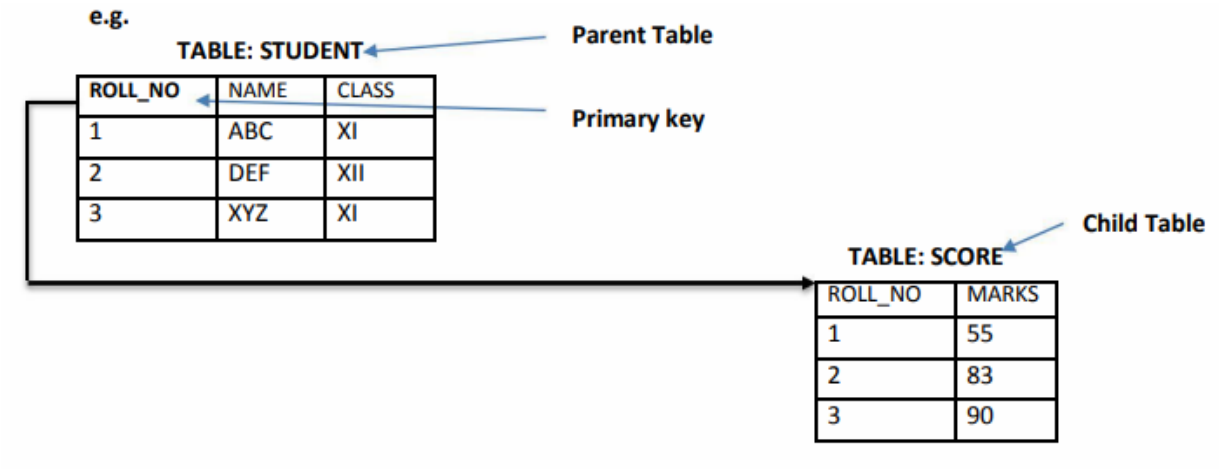
First_Name varchar(30),

PRIMARY KEY (Branch , SID)) ;

FOREIGN KEY CONSTRAINT

- Foreign key is a non key column of a table (child table) that draws its values from primary key of another table(parent table).

- The table in which a foreign key is defined is called a referencing table or child table. A table to which a foreign key points is called referenced table or parent table



ROLL_NO	NAME	CLASS
1	ABC	XI
2	DEF	XII
3	XYZ	XI

ROLL_NO	MARKS
1	55
2	83
3	90

Here column Roll_No is a foreign key in table SCORE (Child Table) and it is drawing its values from

Primary key (ROLL_NO) of STUDENT table.(Parent Key).

CREATE TABLE STUDENT

(ROLL_NO integer NOT NULL PRIMARY KEY ,

NAME VARCHAR(30) ,

CLASS VARCHAR(3));

CREATE TABLE SCORE

(ROLL_NO integer ,MARKS integer ,FOREIGN KEY(ROLL_NO) REFERNCES STUDENT(ROLL_NO)) ;

* Foreign key is always defined in the child table

Syntax for using foreign key

FOREIGN KEY(column name) REFERENCES Parent_Table(PK of Parent Table);

REFERENCING ACTIONS

Referencing action with ON DELETE clause determines what to do in case of a DELETE occurs in the parent table.

Referencing action with ON UPDATE clause determines what to do in case of a UPDATE occurs in the parent table.

Actions:

1. <u>CASCADE</u> : This action states that if a DELETE or UPDATE operation affects a row from the parent table, then Automatically delete or update the matching rows in the child table i.e., cascade the action to child table.

2. <u>SET NULL</u> : This action states that if a DELETE or UPDATE operation affects a row from the parent table, then set the foreign key column in the child table to NULL.

3. <u>NO ACTION</u> : Any attempt for DELETE or UPDATE in parent table is not allowed.

4. <u>RESTRICT</u> : This action rejects the DELETE or UPDATE operation for the parent table.

Q: Create two tables

Customer(customer_id, name)

Customer_sales(transaction_id, amount , customer_id)

Underlined columns indicate primary keys and bold column names indicate foreign key.

Make sure that no action should take place in case of a DELETE or UPDATE in the parent table.

Sol : CREATE TABLE Customer (

customer_id int Not Null Primary Key ,

name varchar(30)) ;

```
CREATE TABLE Customer_sales (

transaction_id Not Null Primary Key ,

amount int ,

customer_id int ,

FOREIGN KEY (customer_id) REFERENCES Customer (customer_id)

ON DELETE NO ACTION

ON UPDATE NO ACTION
```

Conclusion

Assignment No. 4

TITLE:

Perform the following:

For a given set of relation schemes, create tables and perform the following

a. Simple Queries, Simple Queries with Aggregate functions, Queries with Aggregate
b. functions (group by and having clause), Queries involving- Date Functions, String Functions, Math Functions

OBJECTIVE: To demonstrate proficiency in SQL query writing by performing various types of queries on a given set of relation schemes. Learn to create tables based on relation schemes and execute SQL queries involving simple selection, aggregate functions, date functions, string functions, math functions.

Problem Statement:

1. Create Sales table with the following fields(Sales No, Salesname, Branch, Salesamount, DOB)

(a) Insert five records

(b) Calculate total sales amount in each branch

(c) Calculate average sales amount in each branch .

(d) Display all the salesmen, DOB who are born in the month of December as day in character format

i.e. 21-Dec-09

(e) Display the name and DOB of salesman in alphabetical order of the month

(f)Display the list of the Salesman in Upper Case.

Theory:

Aggreagate Funcions:

SQL Aggregate functions are functions where the values of multiple rows are grouped as input on certain criteria to form a single value result of more significant meaning.

It is used to summarize data, by combining multiple values to form a single result.

SQL Aggregate functions are mostly used with the GROUP BY clause of the SELECT statement.

1. Count()
2. Sum()
3. Avg()

4. Min()
5. Max()

Aggregate Functions in SQL

Below is the list of SQL aggregate functions, with examples

Count():

- ***Count(*):*** Returns the total number of records .i.e 6.
- ***Count(salary):*** Return the number of Non-Null values over the column salary. i.e 5.
- ***Count(Distinct Salary):*** Return the number of distinct Non-Null values over the column salary .i.e 4

Sum():

- ***sum(salary):*** Sum all Non-Null values of Column salary i.e., 310
- ***sum(Distinct salary):*** Sum of all distinct Non-Null values i.e., 250.

Avg():

- ***Avg(salary)*** = Sum(salary) / count(salary) = 310/5
- ***Avg(Distinct salary)*** = sum(Distinct salary) / Count(Distinct Salary) = 250/4

Min():

- ***Min(salary):*** Minimum value in the salary column except NULL i.e., 40.

Max():

- ***Max(salary):*** Maximum value in the salary i.e., 80.

Functions :

String Functions

ASCII(): This function is used to find the ASCII value of a character

CHAR_LENGTH(): Doesn't work for SQL Server. Use LEN() for SQL Server. This function is used to find the length of a word.

CHARACTER_LENGTH(): Doesn't work for SQL Server. Use LEN() for SQL Server. This function is used to find the length of a line.

CONCAT(): This function is used to add two words or strings.

CONCAT_WS(): This function is used to add two words or strings with a symbol as concatenating symbol..

FORMAT(): This function is used to display a number in the given format

INSTR(): This function is used to find the occurrence of an alphabet

LCASE(): This function is used to convert the given string into lower case

LOWER(): This function is used to convert the upper case string into lower case.

LPAD(): This function is used to make the given string of the given size by adding the given symbol.

LTRIM(): This function is used to cut the given sub string from the original string.

REVERSE(): This function is used to reverse a string

RIGHT(): This function is used to SELECT a sub string from the right end of the given size

RPAD(): This function is used to make the given string as long as the given size by adding the given symbol on the right.

STRCMP(): This function is used to compare 2 strings.

- If string1 and string2 are the same, the STRCMP function will return 0.
- If string1 is smaller than string2, the STRCMP function will return -1.
- If string1 is larger than string2, the STRCMP function will return 1.

SUBSTR(): This function is used to find a sub string from the a string from the given position.

TRIM(): This function is used to cut the given symbol from the string

UCASE(): This function is used to make the string in upper case

Math Functions

ABS(): It returns the absolute value of a number.

ACOS(): It returns the cosine of a number, in radians.

ASIN(): It returns the arc sine of a number, in radians.

CEIL(): It returns the smallest integer value that is greater than or equal to a number.

COS(): It returns the cosine of a number, in radians.

COT(): It returns the cotangent of a number, in radians.

FLOOR(): It returns the largest integer value that is less than or equal to a number..

MOD(): It returns the remainder (aka. modulus) of n divided by m.

POWER(m, n): It returns m raised to the nth power

ROUND(): It returns a number rounded to a certain number of decimal places.

SQRT(): It returns the square root of a number.

TAN(): It returns the tangent of a number in radians.

Date Functions

DATE()
Extracts the date part of a date or date/time expression.
DATE_ADD()
Adds a specified time interval to a date.
DATE_SUB()
Subtracts a specified time interval from a date. The syntax for DATE_SUB is the same as DATE_ADD just the difference is that DATE_SUB is used to subtract a given interval of date.
DATEDIFF()
Returns the number of days between two dates.

Assignment No. 5

TITLE:

Perform the following:

a. views: creating ,updating and deleting views
b. Join Queries- Inner Join, Outer Join
c. . Sub queries- With IN clause, With EXISTS clause

OBJECTIVE: To demonstrate proficiency in SQL query writing by performing various types of queries on a given set of relation schemes. Learn to create tables based on relation schemes and execute SQL queries involving views, join queries, and subqueries

Theory:

View:

SQL allows a ―virtual relation‖ to be defined by a query, and the relation conceptually contains the result of the query. The virtual relation is not precomputed and stored, but instead is computed by executing the query whenever the virtual relation is used. Any such relation that is not part of the logical model, but is made visible to a user as a virtual relation, is called a view.

Create View: We define a view in SQL by using the create view command.

create view v as <query expression>;

where <query expression> is any legal query expression. The view name is represented by v.

Alter View: The CREATE VIEW statement creates a new view, or replaces an existing view if the OR REPLACE clause is given. If the view does not exist, CREATE OR REPLACE VIEW is the same as CREATE VIEW.

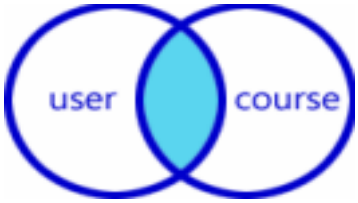
Drop view: Use the DROP VIEW statement to remove a view or an object view from the database. You can change the definition of a view by dropping and re-creating it.

Syntax:

Drop view viewname;

Joins :

INNER JOIN (or just JOIN)



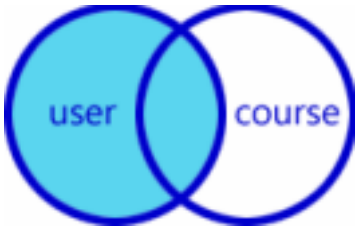
The most frequently used clause is INNER JOIN. This produces a set of records which match in both the user and course tables, i.e. all users who are enrolled on a course:

```
SELECT user.name, course.name
FROM `user`
INNER JOIN `course` on user.course = course.id;
```

Result:

user.name course.name
Alice HTML5
Bob HTML5
Carline CSS3
David MySQL

LEFT JOIN



What if we require a list of all students and their courses even if they‘re not en rolled on one? A LEFT JOIN produces a set of records which matches every entry in the left table (user) regard less of any matching entry in the right table (course):

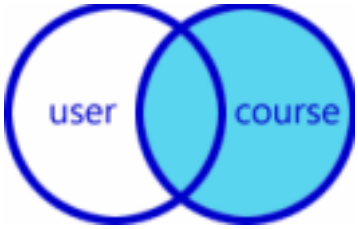
```
SELECT user.name, course.name
FROM `user`
LEFT JOIN `course` on user.course = course.id;
```

Result:

user.name course.name
Alice HTML5
Bob HTML5
Carline CSS3
David MySQL

Emma (NULL)

RIGHT JOIN



Perhaps we require a list all courses and students even if no one has been enrolled? A RIGHT JOIN produces a set of records which matches every entry in the right table (course) regardless of any matching entry in the left table (user):

```
SELECT user.name, course.name
FROM `user`
RIGHT JOIN `course` on user.course = course.id;
```

Joins:

2. **Create following Tables**
   cust_mstr(cust_no,fname,lname)
   add_dets(code_no,add1,add2,state,city,pincode)

   Retrieve the address of customer Fname as 'xyz' and Lname as 'pqr'

   2.Create following Tables
   cust_mstr(custno,fname,lname)
   acc_fd_cust_dets(codeno,acc_fd_no)
   fd_dets(fd_sr_no,amt)

   List the customer holding fixed deposit of amount more than 5000

   3. Create following Tables

   emp_mstr(e_mpno,f_name,l_name,m_name,dept,desg,branch_no)
   branch_mstr(name,b_no)

   List the employee details along with branch names to which they belong
   4. Create following Tables
   emp_mstr(emp_no,f_name,l_name,m_name,dept)
   cntc_dets(code_no,cntc_type,cntc_data)
   List the employee details along with contact details using left outer join & right join

   5. Create following Tables
   cust_mstr(cust_no,fname,lname)
   add_dets(code_no,pincode)
   List the customer who does not have bank branches in their vicinity

RIGHT JOINs are rarely used since you can express the same result using a LEFT JOIN. This can be more effi cient and quicker for the database to parse:

```
SELECT user.name, course.name
FROM `course`
LEFT JOIN `user` on user.course = course.id;
```

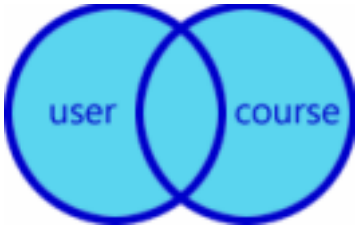
We could, for example, count the number of students enrolled on each course:

```
SELECT course.name, COUNT(user.name)
FROM `course`
LEFT JOIN `user` ON user.course = course.id
GROUP BY course.id;
```

Result:

course.name count()
HTML5 2
CSS3 1
JavaScript 0
PHP 0
MySQL 1

OUTER JOIN (or FULL OUTER JOIN)



Our last option is the OUTER JOIN which returns all records in both tables regard less of any match. Where no match exists, the missing side will contain NULL.

OUTER JOIN is less useful than INNER, LEFT or RIGHT and it‘s not implemented in MySQL. However, you can work around this restriction using the UNION of a LEFT and RIGHT JOIN, e.g.

```
SELECT user.name, course.name
FROM `user`
LEFT JOIN `course` on user.course = course.id

UNION

SELECT user.name, course.name
FROM `user`
RIGHT JOIN `course` on user.course = course.id;
```

Result:

user.name course.name
Alice HTML5
Bob HTML5
Carline CSS3

David MySQL
Emma (NULL)
(NULL) JavaScript
(NULL) PHP

3. Subquery:

In SQL a Subquery can be simply defined as a query within another query. In other words we can say that a Subquery is a query that is embedded in WHERE clause of another SQL query. Important rules for Subqueries:

- You can place the Subquery in a number of SQL clauses: WHERE clause, HAVING clause, FROM clause. Subqueries can be used with SELECT, UPDATE, INSERT, DELETE statements along with expression operator. It could be equality operator or comparison operator such as =, >, =, <= and Like operator.
- A subquery is a query within another query. The outer query is called as **main query** and inner query is called as **subquery**.
- The subquery generally executes first when the subquery doesn't have any **co-relation** with the **main query**, when there is a co-relation the parser takes the decision **on the fly** on which query to execute on **precedence** and uses the output of the subquery accordingly.
- Subquery must be enclosed in parentheses.
- Subqueries are on the right side of the comparison operator.
- ORDER BY command **cannot** be used in a Subquery. GROUPBY command can be used to perform same function as ORDER BY command.
- Use single-row operators with singlerow Subqueries. Use multiple-row operators with multiple-row Subqueries.
  - **Subquery:**
  - 1. emp_mstr(e_mpno,f_name,l_name,m_name,dept,desg,branch_no)
  - Retrieve all employees who are designated as "Manager":
-

Conclusion

Assignment No. 6

TITLE: PL/SQL Programming

Creating Stored Procedure functions and Packages

Error Handling And Exceptions

Triggers and auditing Triggers

Perform the following:

THEORY: PL/SQL

Introduction to PL/SQL,

A procedure (often called a stored procedure) is a subroutine like a subprogram in a regular computing language, stored in database. A procedure has a name, a parameter list, and SQL statement(s). All most all relational database system supports stored procedure, MySQL 5 introduce stored procedure. In the following sections we have discussed MySQL procedure in details and used MySQL 5.6 under Windows 7. MySQL 5.6 supports "routines" and there are two kinds of routines : stored procedures which you call, or functions whose return values you use in other SQL statements the same way that you use pre-installed MySQL functions like pi(). The major difference is that UDFs can be used like any other expression within SQL statements, whereas stored procedures must be invoked using the CALL statement.

Why Stored Procedures?

- Stored procedures are fast. MySQL server takes some advantage of caching, just as prepared statements do. The main speed gain comes from reduction of network traffic. If you have a repetitive task that re quires checking, looping, multiple statements, and no user interaction, do it with a single call to a procedure that's stored on the server.
- Stored procedures are portable. When you write your stored procedure in SQL, you know that it will run on every platform that MySQL runs on, without obliging you to install an additional runtime environment package, or set permissions for program execution in the operating system, or deploy different packages if you have different computer types. That's the advantage of writing in SQL rather than in an external language like Java or C or PHP.

Limitations of SQL

Poor Interface

SQL has a poor interface as it makes look everything very complex even when it's not! Due to its difficult interfacing, users find it difficult to deal with the databases.

2. Cost Inefficient
SQL Server Standard costs around $1,418/year. The high cost makes it difficult for some programmers to use it.

3. Partial Control
SQL doesn't grant the complete control over databases to its users. This is due to some hidden business rules.

4. Security
Regardless of the SQL version, databases in SQL are constantly under threat as it holds huge amounts of sensitive data.

Advantages of PL/SQL

- It is a standard database language and PL/SQL is strongly integrated with SQL. PL/SQL supports both static and also dynamic SQL. Static SQL is said to support DML operations and also the transaction control from PL/SQL block. In dynamic SQL, SQL allows embedding the DDL statements in the PL/SQL blocks.
- Also, It then allows sending an entire block of statements to the database at one time. It reduces network traffic and also provides high performance for the applications.
- It gives high productivity to programmers as it can query, transform and also update the data in the database.
- This is said to save time on the design and also the debugging by strong features, like the exception handling, encapsulation, data hiding and also object-oriented data types.
- Applications that are written in PL/SQL languages are portable.
- this provides high security level.
- It also provides access to the predefined SQL packages.
- It also supports for Object-oriented programming.
- It provides support for developing web applications and server pages.

Syntax and types of PL/SQL blocks

Every PL/SQL statement ends with a semicolon (;). PL/SQL blocks can be nested within other PL/SQL blocks using **BEGIN** and **END**. Following is the basic structure of a PL/SQL block –

```
DECLARE
  <declarations section>
BEGIN
  <executable command(s)>
EXCEPTION
  <exception handling>
END;
```

The 'Hello World' Example

```
DECLARE
  message  varchar2(20):= 'Hello, World!';
BEGIN
  dbms_output.put_line(message);
END;
/
```

The **end;** line signals the end of the PL/SQL block. To run the code from the SQL command line, you may need to type / at the beginning of the first blank line after the last line of the code. When the above code is executed at the SQL prompt, it produces the following result –

Hello World

PL/SQL procedure successfully completed.

Syntax and types of Exception handling:

Following is a general syntax for exception handling:

```
DECLARE
  <declarations section>
BEGIN
  <executable command(s)>
EXCEPTION
  <exception handling goes here >
  WHEN exception1 THEN
     exception1-handling-statements
  WHEN exception2  THEN
    exception2-handling-statements
  WHEN exception3 THEN
    exception3-handling-statements
  ........
  WHEN others THEN
    exception3-handling-statements
END;
```

Assignment 7

TITLE: PL/SQL Programming

Triggers and auditing Triggers

Trigger –

A trigger is a set of SQL statements that reside in system memory with unique names. It is a specialized category of stored procedure that is called automatically when a database server event occurs. Each trigger is always associated with a table.

A **trigger is called a special procedure** because it cannot be called directly like a stored procedure. The key distinction between the trigger and procedure is that a trigger is called automatically when a data modification event occurs against a table. A stored procedure, on the other hand, must be invoked directly.

The following are the main characteristics that distinguish triggers from stored procedures:

- We cannot manually execute/invoked triggers.
- Triggers have no chance of receiving parameters.
- A transaction cannot be committed or rolled back inside a trigger.

Syntax of Trigger

We can create a trigger in SQL Server by using the **CREATE TRIGGER** statement as follows:

CREATE TRIGGER schema.trigger_name
ON table_name
AFTER {**INSERT**, **UPDATE**, **DELETE**}
[NOT **FOR** REPLICATION]
AS
{SQL_Statements}

The parameter descriptions of this syntax illustrate below:

schema: It is an optional parameter that defines which schema the new trigger belongs to.

trigger_name: It is a required parameter that defines the name for the new trigger.

table_name: It is a required parameter that defines the table name to which the trigger applies. Next to the table name, we need to write the AFTER clause where any events like INSERT, UPDATE, or DELETE could be listed.

NOT FOR REPLICATION: This option tells that SQL Server does not execute the trigger when data is modified as part of a replication process.

SQL_Statements: It contains one or more SQL statements that are used to perform actions in response to an event that occurs.

When we use triggers?

Triggers will be helpful when we need to execute some events automatically on certain desirable scenarios. **For example**, we have a constantly changing table and need to know the occurrences of changes and when these changes happen. If the primary table made any changes in such scenarios, we could create a trigger to insert the desired data into a separate table.

Example of Trigger in SQL Server

Let us understand how we can work with triggers in the SQL Server. We can do this by first creating a table named '**Employee'** using the below statements:

Example - all types

```
CREATE TABLE Employee
(
Id INT PRIMARY KEY,
Name VARCHAR(45),
Salary INT,
Gender VARCHAR(12),
DepartmentId INT
)
```

Next, we will insert some record into this table as follows:

```
INSERT INTO Employee VALUES (1,'Steffan', 82000, 'Male', 3),
(2,'Amelie', 52000, 'Female', 2),
```

(3,'Antonio', 25000, 'male', 1),
(4,'Marco', 47000, 'Male', 2),
(5,'Eliana', 46000, 'Female', 3)

We can verify the insert operation by using the SELECT statement. We will get the below output:

1. **SELECT * FROM** Employee;

Id	Name	Salary	Gender	DepartmentId
1	Steffan	82000	Male	3
2	Amelie	52000	Female	2
3	Antonio	25000	male	1
4	Marco	47000	Male	2
5	Eliana	46000	Female	3
6	Peter	62000	Male	3

Conclusion:

Assignment No. 8

TITLE: Cursors(All types)

Write a PL/SQL block that will display the name, dept no, salary of fist highest paid employees.
OR
Write a PL/SQL block that will display the employee details along with salary using cursors.

THEORY:

What is cursor? Cursor types

A cursor in SQL Server is a **d**atabase object that allows us to retrieve each row at a time and manipulate its data**. A curso**r is nothing more than a pointer to a row. It's always used in conjunction with a SELECT statement. It is usually a collection of SQL logic that loops through a predetermined number of rows one by one. A simple illustration of the cursor is when we have an extensive database of worker's records and want to calculate each worker's salary after deducting taxes and leaves.

Cursor Status variables and its use

S.No	Attribute & Description
1	**%FOUND** Returns TRUE if an INSERT, UPDATE, or DELETE statement affected one or more rows or a SELECT INTO statement returned one or more rows. Otherwise, it returns FALSE.
2	**%NOTFOUND** The logical opposite of %FOUND. It returns TRUE if an INSERT, UPDATE, or DELETE statement affected no rows, or a SELECT INTO statement returned no rows. Otherwise, it returns FALSE.
3	**%ISOPEN** Always returns FALSE for implicit cursors, because Oracle closes the SQL cursor automatically after executing its associated SQL statement.
4	**%ROWCOUNT**

	Returns the number of rows affected by an INSERT, UPDATE, or DELETE statement, or returned by a SELECT INTO statement.

Syntax - use of Implicit cursor status variables

```
DECLARE
  total_rows number(2);
BEGIN
  UPDATE customers
  SET salary = salary + 500;
  IF sql%notfound THEN
    dbms_output.put_line('no customers selected');
  ELSIF sql%found THEN
    total_rows := sql%rowcount;
    dbms_output.put_line( total_rows || ' customers selected ');
  END IF;
END;
/
```

When the above code is executed at the SQL prompt, it produces the following result –

6 customers selected

PL/SQL procedure successfully completed.

If you check the records in customers table, you will find that the rows have been updated –

Select * from customers;

```
+----+----------+-----+-----------+----------+
| ID | NAME     | AGE | ADDRESS   | SALARY   |
+----+----------+-----+-----------+----------+
|  1 | Ramesh   |  32 | Ahmedabad |  2500.00 |
|  2 | Khilan   |  25 | Delhi     |  2000.00 |
|  3 | kaushik  |  23 | Kota      |  2500.00 |
|  4 | Chaitali |  25 | Mumbai    |  7000.00 |
|  5 | Hardik   |  27 | Bhopal    |  9000.00 |
|  6 | Komal    |  22 | MP        |  5000.00 |
+----+----------+-----+-----------+----------+
```

Explicit Cursors

Explicit cursors are programmer-defined cursors for gaining more control over the **context area**. An explicit cursor should be defined in the declaration section of the PL/SQL Block. It is created on a SELECT Statement which returns more than one row.

The syntax for creating an explicit cursor is –

CURSOR cursor_name IS select_statement;

Working with an explicit cursor includes the following steps –

- Declaring the cursor for initializing the memory

- Opening the cursor for allocating the memory
- Fetching the cursor for retrieving the data
- Closing the cursor to release the allocated memory

Declaring the Cursor

Declaring the cursor defines the cursor with a name and the associated SELECT statement. For example –

```
CURSOR c_customers IS
  SELECT id, name, address FROM customers;
```

Opening the Cursor

Opening the cursor allocates the memory for the cursor and makes it ready for fetching the rows returned by the SQL statement into it. For example, we will open the above defined cursor as follows –

```
OPEN c_customers;
```

Fetching the Cursor

Fetching the cursor involves accessing one row at a time. For example, we will fetch rows from the above-opened cursor as follows –

```
FETCH c_customers INTO c_id, c_name, c_addr;
```

Closing the Cursor

Closing the cursor means releasing the allocated memory. For example, we will close the above-opened cursor as follows –

```
CLOSE c_customers;
```

Example

Following is a complete example to illustrate the concepts of explicit cursors &minua;

```
DECLARE
  c_id customers.id%type;
  c_name customers.name%type;
  c_addr customers.address%type;
  CURSOR c_customers is
    SELECT id, name, address FROM customers;
BEGIN
  OPEN c_customers;
  LOOP
  FETCH c_customers into c_id, c_name, c_addr;
    EXIT WHEN c_customers%notfound;
    dbms_output.put_line(c_id || ' ' || c_name || ' ' || c_addr);
  END LOOP;
  CLOSE c_customers;
END;
/
```

When the above code is executed at the SQL prompt, it produces the following result –

1 Ramesh Ahmedabad
2 Khilan Delhi
3 kaushik Kota
4 Chaitali Mumbai
5 Hardik Bhopal
6 Komal MP

PL/SQL procedure successfully completed.

Explicit cursor management

1 : Declare Cursor
The first step is to declare the cursor using the below SQL statement:

```
DECLARE cursor_name CURSOR
   FOR select_statement;
```

We can declare a cursor by specifying its name with the data type CURSOR after the DECLARE keyword. Then, we will write the SELECT statement that defines the output for the cursor.

2: Open Cursor
It's a second step in which we open the cursor to store data retrieved from the result set. We can do this by using the below SQL statement:

```
OPEN cursor_name;
```

3: Fetch Cursor
It's a third step in which rows can be fetched one by one or in a block to do data manipulation like insert, update, and delete operations on the currently active row in the cursor. We can do this by using the below SQL statement:

```
FETCH NEXT FROM cursor INTO variable_list;
```

We can also use the @@FETCHSTATUS function in SQL Server to get the status of the most recent FETCH statement cursor that was executed against the cursor. The FETCH statement was successful when the @@FETCHSTATUS gives zero output. The WHILE statement can be used to retrieve all records from the cursor. The following code explains it more clearly:

```
WHILE @@FETCH_STATUS = 0
   BEGIN
      FETCH NEXT FROM cursor_name;
   END;
```

4: Close Cursor
It's a fourth step in which the cursor should be closed after we finished work with a cursor. We can do this by using the below SQL statement:

```
CLOSE cursor_name;
```

5: Deallocate Cursor
It is the fifth and final step in which we will erase the cursor definition and release all the system resources associated with the cursor. We can do this by using the below SQL statement:

DEALLOCATE cursor_name;

CONCLUSION

Assignment No. 9

TITLE: MongoDB Queries using CRUD operations. (Use CRUDoperations, SAVE method, logical operators etc)

OBJECTIVE: To Write the MongoDB Queries using CRUD operations. (Use CRUDoperations, SAVE method, logical operators etc)

PROBLEM STATEMENT:

Write a MongoDB query to display the fields restaurant_id,name, borough and Cuisine for all the documents in the collection restaurant.

Theory:

NOSQL Databases - Explanation, example

NoSQL originally referring to non SQL or non relational is a database that provides a mechanism for storage and retrieval of data.

This data is modeled in means other than the tabular relations used in relational databases. Such databases came into existence in the late 1960s, but did not obtain the NoSQL moniker until a surge of popularity in the early twenty-first century.

NoSQL databases are used in real-time web applications and big data and their use are increasing over time. NoSQL systems are also sometimes called Not only SQL to emphasize the fact that they may support SQL-like query languages.

A NoSQL database includes simplicity of design, simpler horizontal scaling to clusters of machines and finer control over availability. The data structures used by NoSQL databases are different from those used by default in relational databases which makes some operations faster in NoSQL.

Difference between NOSQL and RDBMS

Relational Database	NoSQL
It is used to handle data coming in low velocity.	It is used to handle data coming in high velocity.

It gives only read scalability.	It gives both read and write scalability.
It manages structured data.	It manages all type of data.
Data arrives from one or few locations.	Data arrives from many locations.
It supports complex transactions.	It supports simple transactions.
It has single point of failure.	No single point of failure.
It handles data in less volume.	It handles data in high volume.
Transactions written in one location.	Transactions written in many locations.
Deployed in vertical fashion.	Deployed in Horizontal fashion.

MongoDB – Explanation

MongoDB, the most popular NoSQL database, is an open-source document-oriented database. The term 'NoSQL' means 'non-relational'. It means that MongoDB isn't based on the table-like relational database structure but provides an altogether different mechanism for storage and retrieval of data. This format of storage is called BSON (similar to JSON format).
A simple MongoDB document Structure:

```
{
  title: 'Geeksforgeeks',
  by: 'Harshit Gupta',
  url: 'https://www.geeksforgeeks.org',
  type: 'NoSQL'
}
```

SQL databases store data in tabular format. This data is stored in a predefined data model which is not very much flexible for today's real-world highly growing applications. Modern applications are more networked, social and interactive than ever. Applications are storing more and more data and are accessing it at higher rates.
Relational Database Management System(RDBMS) is not the correct choice when it comes to handling big data by the virtue of their design since they are not horizontally scalable. If the database runs on a single server, then it will reach a scaling limit. NoSQL databases are more scalable and provide superior performance. MongoDB is such a NoSQL database that scales by adding more and more servers and increases productivity with its flexible document model.

TAKE AWAY

- MongoDB is the leading NoSQL database on the market and utilizes a document-oriented database.
- MongoDB supports the rich querying of datasets large or small with fast response times.
- Using dynamic schemas, MongoDB enables agile development and flexibility for application or business requirement changes.
- MongoDB emphasizes performance, scalability, and high availability and in many cases exceeds traditional databases in these areas.
- MongoDB is a proven solution for a wide range of business requirements to companies across numerous industries.

All CRUD Operations - use, syntax, example

As we know that we can use MySQL to use Structure Query Language to store the data in the form of RDBMS.

SQL is the most popular language for adding, accessing, and managing content in a database. It is most noted for its quick processing, proven reliability, ease, and flexibility of use. The application is used for a wide range of purposes,

including data warehousing, e-commerce, and logging applications. The most common use for MySQL, however, is for the purpose of a web database.

1. Create Table Command :
Syntax :

CREATE TABLE table_name (column_name column_type constraints);

Parameters :

1. **column_name –**
   Name of the particular column with any space.
2. **column_type –**
   Datatype of the column. Datatype depends upon the data of the reference column. Datatype can be – char(), varchar(), int(), float(), etc.
3. **constraints –**
   In order to give restrictions to particular column constraints are used. Constraints can be – not null, primary key, foreign key, etc. These are the keywords which give set of restriction to the particular column.

Database – GFG
Table – Student
Student –

- name Varchar(30) NOT NULL
- marks Integer

Example :
use <database> command must be used before any operation on the table.

use gfg;

Create table student(name Varchar(30) NOT NULL, marks Integer);

Output :

Field	Type	Null	Default
name	varchar(30)	No	Null
marks	int(11)	YES	Null

2. Read Operation :
The Read operations are used to retrieve the content of the table from a particular database. Read operation is done by DDL commands.
Example :

use gfg;

select * from student;

name marks
ravi 23
swati 33
kranti12

3. Update Operation :

Altering the content of the table or the structure of the table is done with the help of Update Operations. Two Commands are mostly used for Update Operation –

1. **Alter Table Command –**
   This is the DDL command (Data Definition Language) used to change the structure of the table.

2. **Update Table Command –**
   This is the DML command(Data Manipulating Language) used to alter the records.

Alter Table Command that change the size of name column from varchar(40) to varchar(50) for the Student table :

Alter table student

modify name varchar(50) not null;

Original Table –

desc student;

Field Type Null Default
name varchar(40) YES Null
marks int(11) YES Null

After altering the table –

desc student;

Field Type Null Default
name varchar(50) YES Null
marks int(11) YES Null

Update Command that update the marks of the student from 23 to 100 whose name is ravi using the update command :

Update student set marks = 100

where name = "ravi";

Original Table –

select * from student;

namemarks
ravi 23
swati 33
kranti 12

After updating the table –

select * from student;

namemarks
ravi 100
swati 33
kranti 12

4. Delete Operation :

Two commands are mostly used for the Delete operations –

1. **Delete Command –**
   (DML command) works on the records of the table.

2. **Drop Command –**
   (DDL command) works on the structure of the table.

Delete Command that delete the records of students having marks equal to 100 :

delete from student

where marks = 100;

Original Table –

select * from student;

namemarks
ravi 100
swati 33
kranti 12

After deleting the student records –

select * from student;

name marks
swati 33
kranti 12

Drop Command that drop the table student :

drop table student;

Original Structure –

use gfg;

show tables;

Tables_in_gfg
student

After dropping the student table –

use gfg;
show tables;

Tables_in_gfg

Conclusion:

Group C [DBMS] Mini Project

Develop an application with following details:

1. Follow the same problem statement decided in Assignment-1 of Group A.

2. Follow the Software Development Life cycle and other concepts learnt in Software Engineering Course throughout the implementation.

3. Develop application considering:

Front End: Python/Java/PHP/Perl/Ruby/.NET/ or any other language

Backend : MongoDB/ MySQL/ Oracle / or any standard SQL / NoSQL database

4. Test and validate application using Manual/Automation testing.

5. Student should develop application in group of 2-3 students and submit the Project Report which will consist of documentation related to different phases of Software Development Life Cycle:

Title of the Project, Abstract, Introduction

Software Requirement Specification (SRS)

Conceptual Design using ER features, Relational Model in appropriate Normalize form

Graphical User Interface, Source Code

Testing document

Conclusion.